EduCampus

Documentación Técnica del Sistema

Sistema de Gestión Educativa — Colegio Manantial de Sabiduría (COLMAS)

Camilo Alberto Mendoza Manga
Johnnier Jose Maestre Lopez
Jairo Sebastia Calderon Daza
Reinel Jose Alfaro Rosado

Facultad de Ingenierías y Tecnologías
Universidad Popular del Cesar

Ingeniería de Sistemas

Universidad Popular del Cesar

Septiembre de 2026

Índice

1. Descripción del sistema **3**

1.1. Identificación del problema . . . 3

1.2. Descripción detallada del sistema o aplicación . . . 3

1.3. Modelo de requerimientos . . . 3

1.3.1. Descripción de requisitos funcionales . . . 4

1.3.2. Descripción de requisitos no funcionales . . . 5

1.4. Modelo de casos de uso . . . 6

1.4.1. Diagramas de caso de uso . . . 6

1.4.2. Descripción de caso de uso . . . 6

1.5. Modelo de diseño del sistema . . . 12

1.5.1. Diagrama de clases detallado . . . 12

1.5.2. Diagramas de secuencias . . . 13

1.5.3. Diagrama entidad relación . . . 13

1.5.4. Diagrama de componentes . . . 14

1.6. Producto del software . . . 14

2. Pruebas del software **15**

2.1. Introducción . . . 15

2.2. Planificación de las pruebas . . . 15

2.2.1. Objetivos . . . 15

2.2.2. Alcance . . . 15

2.2.3. Estrategias de pruebas . . . 16

2.2.4. Ambiente de pruebas . . . 16

2.3. Pruebas unitarias . . . 16

2.4. Pruebas de integración . . . 19

2.4.1. Estrategia de pruebas incrementales . . . 19

2.4.2. Estrategia de pruebas basadas en hilos . . . 19

2.4.3. Diseño de casos de pruebas de integración . . . 19
2.5. Pruebas de sistemas . . . 19
2.5.1. Pruebas de seguridad . . . 20
2.5.2. Pruebas de rendimiento . . . 20
2.5.3. Pruebas de usabilidad . . . 21
2.5.4. Pruebas de portabilidad . . . 21
2.5.5. Pruebas de fiabilidad . . . 22
2.5.6. Pruebas de mantenibilidad . . . 22
2.6. Pruebas de aceptación . . . 22
2.6.1. Diseño de caso de pruebas . . . 22
2.6.2. Ejecución y evaluación de la prueba . . . 23

3. Medición del software 24
3.1. Introducción . . . 24
3.1.1. Objetivos . . . 24
3.1.2. Alcance . . . 24
3.1.3. Tipos y herramientas de métricas . . . 24
3.2. Medición del software: atributos internos . . . 24
3.2.1. Métricas de tamaño del código fuente . . . 24
3.2.2. Métricas de tamaño orientadas a clases . . . 25
3.3. Medición de la calidad del software: atributos externos . . . 26

4. Estimación del software 27
4.1. Estimación por punto de función (PF) . . . 27
4.2. Estimación por punto de caso de uso . . . 27
4.3. Estimación por puntos de objetos . . . 27
4.4. Estimación por puntos de historias . . . 27
4.5. Estimación con una herramienta . . . 28
4.6. Análisis de los resultados . . . 28

Conclusiones generales 29

1. Descripción del sistema

1.1. Identificación del problema

El Colegio Manantial de Sabiduría (COLMAS), institución orientada a la educación preescolar y primaria, ha identificado que una parte importante de sus procesos administrativos, académicos y de registro se realiza de forma física y manual. Esta situación resulta poco eficiente, poco estandarizada y difícil de mantener de forma constante, lo que afecta la gestión de la información necesaria para el acompañamiento de los estudiantes.

El proyecto surge para centralizar esos procesos en una solución digital que permita a docentes, personal administrativo y acudientes consultar y registrar información desde un único sistema. El alcance funcional actual comprende estudiantes, cursos, actividades, asistencia, evaluación, convivencia, alertas, materiales, horarios, calendario, notificaciones, boletines y reportes. Los módulos financieros de pagos, pensiones y contabilidad no forman parte del alcance del sistema.

1.2. Descripción detallada del sistema o aplicación

EduCampus es una solución de gestión educativa implementada con un frontend Flutter Web/PWA y un backend REST desarrollado con FastAPI. El backend utiliza SQLAlchemy 2.0 para persistencia en PostgreSQL administrado en Neon, Alembic para migraciones y Pydantic para validación. El frontend utiliza el paquete `http` para consumir la API, `shared_preferences` para persistir localmente el estado elegido de la sesión, y clases *singleton* que extienden `ChangeNotifier` para manejar el estado de los módulos. La aplicación expone tres perfiles principales: Administrador, Docente y Acudiente. El acceso se realiza mediante correo electrónico y contraseña. El estudiante no cuenta con credenciales propias: su información es gestionada y consultada por el administrador, el docente correspondiente y su acudiente.

El Administrador gestiona personas, grados, firmas, evaluación, asistencia, convivencia, alertas, calendario, horarios, formularios, reportes y configuración. El Docente gestiona estudiantes de sus cursos, asistencia, calificaciones o logros, materiales, actividades, entregas, convivencia, horario y avisos de recogida. El Acudiente consulta el progreso del estudiante a su cargo, asistencia, actividades, materiales, horario, calendario, notificaciones y boletines. La API contiene además funcionalidades de matrícula y comunicación, pero algunas de ellas no son consumidas por pantallas activas de la aplicación cliente.

Se adopta Scrum como metodología de referencia, justificada por el desarrollo incremental de módulos como estudiantes, actividades, asistencia, evaluación, seguimiento y comunicación. El flujo de trabajo actual se apoya en integración continua directa sobre la rama principal, despliegue automático y una separación clara entre frontend y backend.

1.3. Modelo de requerimientos

1.3.1. Descripción de requisitos funcionales

ID	Módulo	Requisito	Actor/capa	Estado
RF01	Autenticación	Iniciar sesión con correo y contraseña mediante `/auth/login/`.	Administrador, Docente, Acudiente	Implementado
RF02	Sesión	Restaurar la sesión local mediante `/auth/session/`.	Todos	Implementado
RF03	Usuarios	Registrar estudiantes, docentes y acudientes mediante `/auth/registro/`.	Administrador	Implementado
RF04	Usuarios	Consultar estudiantes, docentes y acudientes.	Administrador	Implementado
RF05	Grados	Crear, editar y eliminar grados.	Administrador	Implementado
RF06	Firmas	Consultar/cargar firmas digitales de docentes y administrador.	Administrador	Implementado
RF07	Evaluación	Registrar calificaciones numéricas de 0 a 5.	Docente, Administrador	Implementado
RF08	Evaluación	Registrar logros/observaciones de preescolar por periodo.	Docente, Administrador	Implementado
RF09	Boletines	Generar y descargar boletines PDF por estudiante y periodo.	Acudiente, Administrador	Implementado
RF10	Asistencia	Registrar y consultar asistencia de estudiantes.	Docente, Administrador	Implementado
RF11	Asistencia docente	Registrar y consultar asistencia de profesores.	Administrador	Implementado
RF12	Convivencia	Registrar, consultar, actualizar seguimiento y eliminar situaciones.	Docente, Administrador	Implementado
RF13	Alertas	Crear y atender alertas con respuesta administrativa.	Docente, Administrador	Implementado
RF14	Recogida	Registrar finalización de clase y avisar a acudientes.	Docente	Implementado
RF15	Notificaciones	Consultar notificaciones y marcarlas como leídas.	Docente, Acudiente	Implementado
RF16	Materiales	Publicar, listar y eliminar material didáctico.	Docente, Administrador	Implementado
RF17	Actividades	Crear y listar tareas con fecha límite y opción de video.	Docente, Acudiente	Implementado
RF18	Entregas	Subir entregas *multipart*, descargar archivos y calificarlas.	Acudiente, Docente	Implementado

ID	Módulo	Requisito	Actor/capa	Estado
RF19	Reglas de entrega	Rechazar entregas fuera de plazo, formatos no permitidos, video no autorizado y archivos > 200 MiB.	Backend	Implementado
RF20	Horario	Crear, listar y eliminar bloques de horario.	Administrador, Docente, Acudiente	Implementado
RF21	Calendario	Crear, consultar y eliminar eventos.	Administrador, Todos consulta	Implementado
RF22	Comunicación	Crear comunicados institucionales.	Administrador	Endpoint existe; frontend activo no persiste
RF23	Formularios	Crear plantillas, campos y diligenciar formularios.	Administrador, usuarios según formulario	Parcial
RF24	Campos personalizados	Definir campos adicionales en la ficha del estudiante.	Administrador	Implementado en frontend/modelo
RF25	Reportes	Generar reporte diario y estadísticas institucionales.	Administrador	Implementado
RF26	PWA	Instalar la aplicación web como PWA.	Todos	Implementado
RF27	Salud	Verificar API y conexión a PostgreSQL mediante `/` y `/health`.	Operación	Implementado
RF28	Matrícula	Crear matrícula mediante `/matriculas/`.	Administrador	Endpoint disponible; no hay interfaz activa

Tabla 1: Requisitos funcionales de EduCampus

1.3.2. Descripción de requisitos no funcionales

ID	Categoría	Requisito	Estado
RNF01	Seguridad	Contraseñas verificadas con *bcrypt*; existe además cifrado reversible para consulta administrativa.	Implementado; riesgo residual por reversibilidad
RNF02	Sesiones	Sesiones del backend se mantienen en memoria y se invalidan al reiniciar.	Limitación actual
RNF03	Disponibilidad	Backend desplegado en Render gratuito, con suspensión por inactividad.	Limitación actual
RNF04	Persistencia	PostgreSQL gestionado en Neon.	Implementado
RNF05	Archivos	Entregas en *filesystem* de Render; el plan gratuito usa almacenamiento efímero.	Limitación actual

ID	Categoría	Requisito	Estado
RNF06	Validación	Pydantic valida dominios de correo, contraseña, rol, nota, periodo y otros *payloads*.	Implementado
RNF07	API	Comunicación REST en JSON y `multipart/form-data`.	Implementado
RNF08	Portabilidad	Flutter Web/PWA; prueba responsive para 320×568, 390×844, 768×1024 y 1366×768.	Implementado en código/pruebas
RNF09	Mantenibilidad	Separación del frontend en `models`, `services`, `screens`, `widgets` y `core`; y del backend en `main`, `models`, `schemas` y `database`.	Implementado
RNF10	Integridad	FK, UNIQUE y políticas `ondelete` definidas en SQLAlchemy.	Implementado
RNF11	CORS	`allow_origins=["*"]` en el backend.	Implementado; configuración permisiva
RNF12	Rendimiento	Cliente usa *timeout* de 45 s en operaciones HTTP relevantes. No existe SLA formal.	Parcial
RNF13	Calidad visual	La suite responsive abre paneles y opciones en cuatro resoluciones y comprueba ausencia de excepciones.	Implementado en pruebas

Tabla 2: Requisitos no funcionales de EduCampus

1.4. Modelo de casos de uso

1.4.1. Diagramas de caso de uso

Imagen pendiente

`img/casos_uso.png`

Figura 1: Diagrama de casos de uso de EduCampus

1.4.2. Descripción de caso de uso

CU01 — Autenticar usuario

Campo	Descripción
ID	CU01
Nombre	Autenticar usuario
Actor	Administrador/Docente/Acudiente
Prerrequisitos	Cuenta registrada y API disponible.
Flujo	1) Ingresar correo y contraseña. 2) Validar UI. 3) `POST /auth/login/`. 4) Buscar cuenta. 5) Verificar *bcrypt*. 6) Crear sesión. 7) Dirigir al panel.
Excepciones	401 por credenciales; error de conexión; rol sin destino.

CU02 — Restaurar sesión

Campo	Descripción
ID	CU02
Nombre	Restaurar sesión
Actor	Todos
Prerrequisitos	Token local existente.
Flujo	1) Leer `SharedPreferences`. 2) `GET /auth/session/`. 3) Si es válida, reconstruir `SesionUsuario`. 4) Navegar al destino.
Excepciones	Token inválido o servidor reiniciado: limpiar sesión y volver a login.

CU03 — Gestionar estudiante

Campo	Descripción
ID	CU03
Nombre	Gestionar estudiante
Actor	Administrador
Prerrequisitos	Sesión de administrador.
Flujo	Consultar, registrar y revisar estudiantes.
Excepciones	La edición de un registro existente no persiste actualmente.

CU04 — Gestionar evaluación

Campo	Descripción
ID	CU04
Nombre	Gestionar evaluación
Actor	Docente/Administrador
Prerrequisitos	Estudiante y curso válidos.
Flujo	Registrar calificación 0..5 o logro/observación según grado y consultar.
Excepciones	Payload inválido: 422; periodo fuera de I–IV: 422.

CU05 — Registrar asistencia

Campo	Descripción
ID	CU05
Nombre	Registrar asistencia
Actor	Docente/Administrador
Prerrequisitos	Estudiante/curso existentes.
Flujo	Registrar estado para fecha no futura.
Excepciones	Fecha futura: 400.

CU06 — Gestionar actividad

Campo	Descripción
ID	CU06
Nombre	Gestionar actividad
Actor	Docente
Prerrequisitos	Curso existente.
Flujo	Crear tarea, descripción, fecha límite y permiso de video.
Excepciones	Datos faltantes: 422; curso inexistente: 404.

CU07 — Entregar actividad

Campo	Descripción
ID	CU07
Nombre	Entregar actividad
Actor	Acudiente
Prerrequisitos	Actividad vigente; estudiante y acudiente válidos.
Flujo	*Multipart*: archivo + comentario; validar plazo/tipo/tamaño; guardar; notificar docente.
Excepciones	400 por plazo/formato/video/tamaño; 404 por IDs; duplicado por UNIQUE.

CU08 — Calificar entrega

Campo	Descripción
ID	CU08
Nombre	Calificar entrega
Actor	Docente
Prerrequisitos	Entrega existente.
Flujo	Registrar nota y retroalimentación.
Excepciones	404 si no existe; 422 por nota fuera de rango.

CU09 — Gestionar convivencia

Campo	Descripción
ID	CU09
Nombre	Gestionar convivencia
Actor	Docente/Administrador
Prerrequisitos	Estudiante existente.
Flujo	Crear situación, consultar, actualizar seguimiento o eliminar.
Excepciones	404 si id inexistente.

CU10 — Atender alerta

Campo	Descripción
ID	CU10
Nombre	Atender alerta
Actor	Administrador
Prerrequisitos	Alerta existente.
Flujo	Marcar atendida y guardar respuesta.
Excepciones	404 si no existe; 401/403 según sesión.

CU11 — Avisar recogida

Campo	Descripción
ID	CU11
Nombre	Avisar recogida
Actor	Docente
Prerrequisitos	Curso existente.
Flujo	Registrar clase finalizada, crear notificaciones a acudientes y preparar WhatsApp.
Excepciones	Curso inexistente; estudiante/acudiente sin relación.

CU12 — Generar boletín

Campo	Descripción
ID	CU12
Nombre	Generar boletín
Actor	Administrador/Acudiente
Prerrequisitos	Estudiante y firmas requeridas.
Flujo	`GET /estudiantes/{id}/boletin/`; el backend construye el PDF con ReportLab.
Excepciones	No generar si falta firma requerida.

CU13 — Gestionar materiales

Campo	Descripción
ID	CU13
Nombre	Gestionar materiales
Actor	Docente/Administrador/Acudiente
Prerrequisitos	Curso existente.
Flujo	Subir/listar/eliminar o consultar material.
Excepciones	Formato/tamaño de archivo según endpoint.

CU14 — Gestionar horario

Campo	Descripción
ID	CU14
Nombre	Gestionar horario
Actor	Administrador/Docente/Acudiente
Prerrequisitos	Curso existente.
Flujo	Crear/listar/eliminar bloque por día y hora.
Excepciones	Duplicidades o datos inválidos.

CU15 — Gestionar calendario

Campo	Descripción
ID	CU15
Nombre	Gestionar calendario
Actor	Administrador/usuarios consulta
Prerrequisitos	Sesión.
Flujo	Crear/listar/eliminar eventos.
Excepciones	Id inexistente.

CU16 — Consultar reportes

Campo	Descripción
ID	CU16
Nombre	Consultar reportes
Actor	Administrador
Prerrequisitos	Sesión de administrador.
Flujo	Consultar estadísticas y reporte diario de asistencia.
Excepciones	Error de BD / filtros inválidos.

1.5. Modelo de diseño del sistema

El diseño se documenta mediante clases del dominio, secuencias, el modelo entidad-relación y los componentes que conectan el frontend con la API y PostgreSQL.

1.5.1. Diagrama de clases detallado

El frontend organiza el dominio en modelos Dart y servicios `ChangeNotifier` *singleton*. El backend usa clases SQLAlchemy como entidades persistentes y esquemas Pydantic para validación.

Imagen pendiente

`img/diagrama_clases.png`

Figura 2: Diagrama de clases detallado

1.5.2. Diagramas de secuencias

Flujo de autenticación:



Figura 3: Diagrama de secuencia del flujo de autenticación

1.5.3. Diagrama entidad relación

La implementación ORM actual contiene 21 clases persistentes, que corresponden a las tablas de la base de datos.

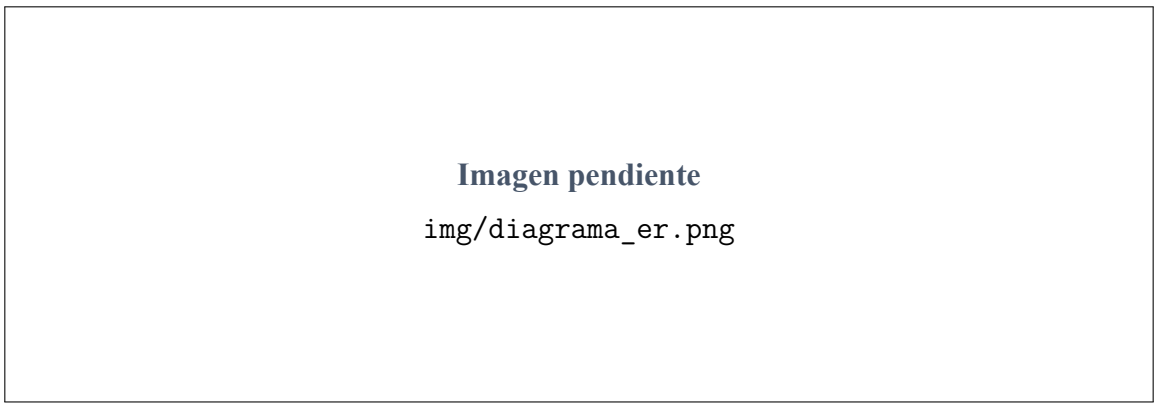


Figura 4: Diagrama entidad-relación

1.5.4. Diagrama de componentes



Figura 5: Diagrama de componentes

1.6. Producto del software

EduCampus se encuentra desplegado en producción y su código fuente está disponible en los siguientes recursos:

- **Frontend (producción):** https://edu-campus-app-frontend.vercel.app
- **Backend (API):** https://educampus-backend-rwz6.onrender.com
- **Repositorio del frontend:** https://github.com/Reinel-ing/EduCampus_app_frontend
- **Repositorio del backend:** https://github.com/Reinel-ing/EduCampus_app_backend

2. Pruebas del software

2.1. Introducción

Esta unidad estructura la verificación de EduCampus en niveles unitario, integración, sistema y aceptación. Se distingue entre pruebas automatizadas ya presentes en el repositorio y pruebas diseñadas en este documento que requieren ejecución real. Ningún resultado no observado se presenta como aprobado.

2.2. Planificación de las pruebas

2.2.1. Objetivos

Objetivo general: verificar la corrección funcional e integración de EduCampus y detectar fallas en reglas de negocio críticas antes de la aceptación del producto.

Objetivos específicos:

- Verificar autenticación, restauración y cierre de sesión.
- Validar límites y clases de entrada de evaluación, periodos, contraseñas, archivos y asistencia.
- Comprobar la regla de fecha límite de entregas y la restricción de fechas futuras en asistencia.
- Comprobar generación de boletines y dependencia de firmas digitales.
- Validar la integración frontend ↔ API ↔ PostgreSQL y los flujos críticos por rol.
- Identificar riesgos de seguridad, disponibilidad, fiabilidad, portabilidad y mantenibilidad.

2.2.2. Alcance

Módulo a probar	Objetivos
Autenticación	Login, sesión, logout, restauración, recuperación.
Usuarios	Registro, consulta y autorización.
Evaluación	Notas/logros y periodos.
Asistencia	Estudiantes/docentes y fechas.
Actividades y entregas	Plazo, formato, video, tamaño, duplicidad, calificación.
Boletines	Firmas y PDF.
Convivencia/alertas	Registro, seguimiento y atención.
Notificaciones/recogida	Creación y consulta.

Módulo a probar	Objetivos
Responsive/PWA	Cuatro resoluciones de pruebas existentes.

Tabla 3: Alcance de las pruebas por módulo

2.2.3. Estrategias de pruebas

Nivel	Técnicas	Modelos/estrategias	Herramientas
Pruebas unitarias	Equivalencia, límites, camino básico	Caja negra/caja blanca	`flutter_test`; validadores Pydantic
Pruebas de integración	Incremental ascendente/descendente; hilos	Frontend-API-BD; API-archivo-notificación	Flutter test; Postman/cURL
Pruebas de sistemas	Seguridad, rendimiento, usabilidad, portabilidad, fiabilidad, mantenibilidad	*End-to-end*	OWASP ZAP; JMeter; navegador; SonarQube
Pruebas de aceptación	Escenarios de negocio	Flujos por rol	Selenium/Katalon

Tabla 4: Estrategias de prueba por nivel

2.2.4. Ambiente de pruebas

Elemento	Configuración observada/definida
Frontend	Flutter Web/PWA; Dart ^3.13.2; `http` 1.2.0; `shared_preferences` 2.5.5.
Backend	FastAPI 0.141.1 + Uvicorn.
Persistencia	PostgreSQL + SQLAlchemy 2.0.54 + psycopg2.
Validación	Pydantic 2.13.5.
Migraciones	Alembic 1.20.0.
Documentos	ReportLab 5.0.1.
Resoluciones responsive	320×568, 390×844, 768×1024, 1366×768.
Hardware	Registrar el equipo real usado en la ejecución final.

Tabla 5: Ambiente de pruebas

2.3. Pruebas unitarias

El repositorio ya incluye dos pruebas automatizadas de interfaz:

- `widget_test.dart`: arranque de la aplicación.

- `responsive_test.dart`: Login y paneles de Administrador, Docente y Acudiente, recorriendo las opciones disponibles en cuatro resoluciones.

Análisis de clases de equivalencia

Condición	Clase válida	Clase no válida	Casos
Correo	Email + gmail	Formato/dominio inválido	CP-002/003
Contraseña	6..100 + mayúscula + número + especial	Fuera de rango/reglas	CP-001/002
Nota	0..5	< 0 o > 5	CP-007..009
Periodo	I..IV	Otro	CP-008..010
Asistencia	Fecha $\leq$ hoy	Fecha futura	CP-011/012
Archivo	Extensión válida y $\leq$ 200 MiB	Extensión/tamaño inválido	CP-013..017

Tabla 6: Clases de equivalencia

Casos por valores límite

Variable	Inferior	Exacto	Superior	Criterio
Contraseña	5	6	7	5 falla; 6 pasa.
Contraseña	99	100	101	100 pasa; 101 falla.
Calificación	−0,01	0	0.01	0 pasa; negativo falla.
Calificación	4.99	5	5.01	5 pasa; 5.01 falla.
Archivo	199 MiB	200 MiB	200 MiB + 1	200 pasa; +1 falla.
Fecha asistencia	Ayer	Hoy	Mañana	Hoy pasa; mañana falla.
Periodo	III	I o IV	V	I y IV pasan; V falla.
Ancho responsive	899	900	901	899 usa menú móvil; 900 no.

Tabla 7: Casos por valores límite

Camino básico

Método real analizado: `AuthService.iniciarSesion()`, ubicado en:

`frontend/lib/services/servicio_auth.dart`

```
Future<SesionUsuario> iniciarSesion({
  required String correo,
  required String password,
}) async {
  final uri = Uri.parse('${ApiConfig.baseUrl}/auth/login/');
```

```
  http.Response respuesta;
  try {
    respuesta = await http.post(
      uri,
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'correo': correo.trim().toLowerCase(),
        'password': password,
      }),
    ).timeout(const Duration(seconds: 45));
  } catch (_) {
    throw const AuthException('No fue posible conectar con el servidor.');
  }
  if (respuesta.statusCode == 200) {
    final datos = jsonDecode(utf8.decode(respuesta.bodyBytes));
    final sesion = SesionUsuario(
      accessToken: datos['access_token'] as String,
      rol: datos['rol'] as String,
      nombre: datos['nombre'] as String,
      usuarioId: datos['usuario_id'] as int,
      correo: datos['correo'] as String,
    );
    _sesionActual = sesion;
    notifyListeners();
    return sesion;
  }
  if (respuesta.statusCode == 401) {
    throw const AuthException('Correo o contraseña incorrectos');
  }
  throw const AuthException(
    'No fue posible iniciar sesión. Intenta de nuevo más tarde.',
  );
}
```

Listing 1: Método `AuthService.iniciarSesion()`

La complejidad ciclomática es $V(G) = 4$, por tres predicados de decisión.

Camino	Secuencia	Datos	Resultado
P1	Inicio → URI → POST → Excepción → Error de conexión → Fin	Servidor inaccesible / *timeout*	Se prueba el `catch`.
P2	Inicio → URI → POST → Sin excepción → 200 → Crear sesión → Notificar → Retornar → Fin	HTTP 200 + JSON válido	Se crea `SesionUsuario`.
P3	Inicio → URI → POST → Sin excepción → No 200 → 401 → Error de credenciales → Fin	HTTP 401	`AuthException` específica.
P4	Inicio → URI → POST → Sin excepción → No 200 → No 401 → Error genérico → Fin	HTTP 500, 503 u otro	`AuthException` genérica.

Tabla 8: Caminos independientes de `iniciarSesion()`

Ejecución y evaluación: los casos que requieren *backend*, PostgreSQL, Render o herramientas externas se registran como *Pendiente de ejecución*.

2.4. Pruebas de integración

2.4.1. Estrategia de pruebas incrementales

ID	Integración	Estrategia	Caso	Resultado esperado
I01	`AuthService` ↔ `/auth/login/`	Ascendente	200/401/*timeout*	`SesionUsuario` correcta o error controlado.
I02	API auth ↔ PostgreSQL	Descendente	Cuenta existente / inexistente	Rol y *bcrypt* correctos.
I03	Tarea ↔ Entrega ↔ archivo ↔ notificación	Incremental / Hilo	Válida/vencida	Reglas y persistencia coordinadas.
I04	Calificaciones ↔ observaciones ↔ boletín ↔ firmas	Incremental / Hilo	Completo / incompleto	PDF solo cuando se cumplen reglas.

Tabla 9: Pruebas de integración incrementales

2.4.2. Estrategia de pruebas basadas en hilos

- **Hilo 1 — Autenticación:** Login → buscar cuenta → *bcrypt* → crear sesión → panel.
- **Hilo 2 — Entrega:** consultar tarea → *multipart* → validar plazo/extensión/tamaño → guardar archivo → `INSERT` → notificar.
- **Hilo 3 — Boletín:** consultar periodo → calificaciones/observaciones → firmas → ReportLab → PDF.

2.4.3. Diseño de casos de pruebas de integración

Los casos I01–I04 se combinan con los casos formales CP-001..CP-022.

Ejecución y evaluación de las pruebas de integración

Pendiente de ejecución real y evidencia de herramienta.

2.5. Pruebas de sistemas

2.5.1. Pruebas de seguridad

Campo	Contenido
Tipo	Seguridad
Nombre	Protección de autenticación
Descripción	Comprobar que contraseñas incorrectas no generan sesión y que *endpoints* protegidos rechazan tokens inválidos.
Ambiente / condiciones	API levantada + PostgreSQL de pruebas; cuentas ficticias.
Herramientas / metodología	Pruebas API + revisión estática; OWASP ZAP como herramienta propuesta.
Diseño	Diseño listo; ejecución externa requerida.
Resultado	No ejecutado en este entorno.

2.5.2. Pruebas de rendimiento

Campo	Contenido
Tipo	Rendimiento
Nombre	Respuesta de operaciones críticas
Descripción	Medir latencia de login, listado de tareas, carga de entrega y boletín bajo carga controlada.
Ambiente / condiciones	Ambiente *staging*; datos representativos.
Herramientas / metodología	Apache JMeter.
Diseño	Diseño listo; ejecución externa requerida.
Resultado	No ejecutado; no se inventan tiempos.

2.5.3. Pruebas de usabilidad

Campo	Contenido
Tipo	Usabilidad
Nombre	Navegación por rol
Descripción	Comprobar que los tres paneles abren opciones sin excepciones y que formularios muestran validaciones comprensibles.
Ambiente / condiciones	Chrome/Edge; resoluciones definidas en `responsive_test.dart`.
Herramientas / metodología	Flutter *widget tests* + inspección manual.
Diseño	Hay prueba automática de desbordes.
Resultado	Pendiente de ejecución completa.

2.5.4. Pruebas de portabilidad

Campo	Contenido
Tipo	Portabilidad
Nombre	PWA y tamaños
Descripción	Verificar que la interfaz funciona en teléfono, tablet y escritorio; la suite actual cubre cuatro tamaños.
Ambiente / condiciones	320×568, 390×844, 768×1024, 1366×768.
Herramientas / metodología	`flutter_test` + navegador.
Diseño	Hay prueba automática de cuatro tamaños.
Resultado	Ejecución no realizada aquí.

2.5.5. Pruebas de fiabilidad

Campo	Contenido
Tipo	Fiabilidad
Nombre	Restauración de sesión y BD
Descripción	Comprobar recuperación cuando el *backend* reinicia y cuando la BD pierde conectividad temporalmente.
Ambiente / condiciones	*Backend staging* + PostgreSQL.
Herramientas / metodología	Pruebas de integración/API.
Diseño	Diseño de casos.
Resultado	Pendiente.

2.5.6. Pruebas de mantenibilidad

Campo	Contenido
Tipo	Mantenibilidad
Nombre	Migraciones y estructura
Descripción	Verificar que Alembic actualiza el esquema y que los servicios del frontend pueden modificarse sin romper pantallas no relacionadas.
Ambiente / condiciones	Repositorio completo.
Herramientas / metodología	Revisión estática + ejecución de migraciones.
Diseño	13 revisiones Alembic presentes.
Resultado	Pendiente de *pipeline* automatizado.

2.6. Pruebas de aceptación

2.6.1. Diseño de caso de pruebas

ID	Requisito asociado	Escenario	Datos de entrada	Pasos	Resultado esperado	Resultado obtenido
CA001	RF01	Ingreso exitoso	Correo / contraseña válidos	Abrir app → ingresar → validar	Acceso al panel	Pendiente
CA002	RF07	Docente registra nota válida	`score=4.5`, periodo=II	Abrir evaluación → guardar	Nota visible / persistida	Pendiente

ID	Requisito asociado	Escenario	Datos de entrada	Pasos	Resultado esperado	Resultado obtenido
CA003	RF12	Registrar convivencia	Situación positiva	Abrir convivencia → registrar	Situación visible y seguimiento disponible	Pendiente
CA004	RF18 / RF19	Acudiente entrega tarea en plazo	PDF ≤ 200 MiB	Abrir tarea → adjuntar → enviar	Entrega creada	Pendiente
CA005	RF09	Descargar boletín	`student_id` + firmas	Abrir boletín → descargar	PDF generado	Pendiente
CA006	RF14	Aviso de recogida	Curso con acudientes	Finalizar clase → avisar	Notificaciones creadas	Pendiente

Tabla 10: Casos de prueba de aceptación

2.6.2. Ejecución y evaluación de la prueba

Se propone ejecutar estos casos con Selenium o Katalon, conservando las evidencias de cada corrida. A la fecha no se ha realizado una ejecución con estas herramientas; por tanto, el resultado se deja *Pendiente de ejecución*.

3. Medición del software

3.1. Introducción

La medición establece una línea base cuantitativa del producto y apoya el diagnóstico técnico. Se miden atributos internos y externos; para estos últimos se toma como referencia ISO/IEC 25010:2023, la versión vigente del modelo de calidad de producto.

3.1.1. Objetivos

Medir tamaño y estructura del código; identificar complejidad y acoplamiento en clases ORM; evaluar indicadores de usabilidad, seguridad, portabilidad, mantenibilidad, rendimiento, fiabilidad, compatibilidad y adecuación funcional; y disponer de una línea base para versiones posteriores.

3.1.2. Alcance

Se incluyen `frontend/lib`, el *backend* Python (excluyendo `venv`), las pruebas Flutter, los modelos ORM y los *endpoints* FastAPI. No se incluyen dependencias descargadas.

3.1.3. Tipos y herramientas de métricas

Se usan *scripts* de conteo y análisis AST para las métricas actuales; SonarQube queda como herramienta propuesta, pero no fue ejecutada en este entorno; y se emplea la revisión de pruebas para los indicadores externos.

3.2. Medición del software: atributos internos

3.2.1. Métricas de tamaño del código fuente

Métrica	Método	Resultado	Análisis
LOC frontend Dart (sin blancos)	Cuenta de líneas no blancas de `frontend/lib`	14.611	Volumen de código cliente medido directamente.
LOC backend Python (sin blancos)	Cuenta de líneas no blancas excluyendo `backend/venv`	4.446	Volumen de código servidor medido directamente.
LOC total fuente (sin blancos)	Dart + Python	19.057	Tamaño combinado de las capas principales.
Archivos Dart productivos	`find frontend/lib/**/*.dart`	92	Fragmentación del frontend.

Métrica	Método	Resultado	Análisis
Pantallas Flutter	`find frontend/lib/screens/**/*.dart`	38	Cobertura de UI por módulo.
Servicios Flutter	`find frontend/lib/services/**/*.dart`	27	Componentización del acceso a datos/estado.
Modelos Dart	`find frontend/lib/models/**/*.dart`	16	Modelado del dominio cliente.
Widgets reutilizables	`find frontend/lib/widgets/**/*.dart`	7	Reutilización de UI.
Clases ORM SQLAlchemy	AST de `models.py`	21	Entidades persistentes activas en el código actual.
Endpoints FastAPI	AST de `main.py`	62	Superficie funcional de la API.
Revisiones Alembic	`alembic/versions/*.py`	13	Evolución del esquema.
Pruebas Flutter	`frontend/test/*.dart`	2	Pruebas automatizadas presentes.

Tabla 11: Métricas de tamaño del código fuente

Estos conteos son reproducibles y excluyen `backend/venv`. No equivalen a una medición de SonarQube.

3.2.2. Métricas de tamaño orientadas a clases

Clase	Métodos / WMC	DIT	CBO aprox.	Cohesión
Administrador	0	1	0	Alta
Grado	0	1	2	Alta
Acudiente	0	1	0	Alta
Estudiante	0	1	2	Alta
Profesor	0	1	0	Alta
Curso	0	1	2	Alta
Matricula	0	1	2	Alta
Calificacion	0	1	2	Alta
Observacion	0	1	2	Alta
Asistencia	0	1	2	Alta
AsistenciaDocente	0	1	2	Alta
Convivencia	0	1	1	Alta
AlertaAlumno	0	1	1	Alta
Notificacion	0	1	5	Media
MaterialDidactico	0	1	1	Alta
Tarea	0	1	1	Alta
TareaEntrega	0	1	5	Media

Clase	Métodos / WMC	DIT	CBO aprox.	Cohesión
Horario	0	1	1	Alta
Comunicado	0	1	0	Alta
EventoCalendario	0	1	1	Alta
ClaseFinalizada	0	1	2	Alta

Tabla 12: Métricas orientadas a clases del modelo ORM

WMC es el número de métodos definidos; DIT vale 1 por herencia directa de `Base`; CBO se aproxima por las referencias en `relationship` y `ForeignKey`. La cohesión se expresa de forma cualitativa porque el código ORM es principalmente declarativo.

3.3. Medición de la calidad del software: atributos externos

Característica	Métrica	Fórmula/criterio	Resultado	Análisis
Usabilidad	Cobertura responsive	4 resoluciones verificadas en el código de pruebas / 4 previstas	100 % del conjunto definido en `responsive_test.dart`	La suite recorre Login y los paneles por rol.
Portabilidad	Resoluciones cubiertas	4	4	320×568; 390×844; 768×1024; 1366×768.
Seguridad	Validaciones de entrada	Correo, contraseña, rol, *score*, periodo y archivos	Cobertura definida en `schemas.py` y `main.py`	La cobertura debe complementarse con pruebas negativas y escaneo.
Fiabilidad	Rutas de salud	2 *endpoints*: `/` y `/health`	2/2 documentados	`/health` consulta `SELECT 1` en PostgreSQL.
Compatibilidad	Capas cliente-servidor	Flutter + HTTP REST + FastAPI + JSON / *multipart*	1 integración principal	No usa acceso directo del cliente a PostgreSQL.
Adecuación funcional	Requisitos funcionales documentados	28	28 trazados al código / *endpoints* o limitaciones	RF22, RF23 y RF28 tienen estado parcial o sin frontend activo.

Tabla 13: Métricas de calidad externa según ISO/IEC 25010:2023

ISO/IEC 25010:2023 es el modelo de calidad de producto usado como referencia conceptual para los atributos evaluados.

4. Estimación del software

4.1. Estimación por punto de función (PF)

Se realiza una estimación académica preliminar. Supuestos: 28 EI, 27 EQ, 7 EO, 21 ILF, 0 EIF; pesos medios 4/4/5/10. UFP = 465. Usando 10 h/PF como supuesto de productividad únicamente para comparar magnitudes, se obtienen 4.650 h (≈ 29,1 persona-mes a 160 h/mes). No representa el esfuerzo real del equipo.

Componente	Cantidad	Peso medio	Subtotal
EI	28	4	112
EQ	27	4	108
EO	7	5	35
ILF	21	10	210
EIF	0	7	0
UFP			**465**

Tabla 14: Cálculo de puntos de función sin ajustar (UFP)

4.2. Estimación por punto de caso de uso

Se consideran 3 actores humanos y 16 casos de uso principales. Con 3 puntos por actor complejo y 10 por caso de uso medio: UAW = 9, UUCW = 160, UUCP = 169. Con 20 h/PCU se obtienen 3.380 h (≈ 21,1 persona-mes). Es una aproximación académica sin ajuste técnico/ambiental por falta de datos históricos.

4.3. Estimación por puntos de objetos

Conteos observados: 38 pantallas, 27 servicios y 16 modelos Dart. Con pesos referenciales 2, 1 y 2: PO = 135. Con 8 h/PO se obtienen 1.080 h (≈ 6,8 persona-mes).

4.4. Estimación por puntos de historias

Épica / módulo	Story Points
Autenticación y sesión	13
Gestión de usuarios y grados	13
Evaluación y boletines	18
Asistencia	11
Convivencia y alertas	13
Actividades y entregas	18

Épica / módulo	Story Points
Materiales	8
Horario y calendario	8
Notificaciones y recogida	13
Formularios y campos	13
Reportes y configuración	8
PWA y portabilidad	6
QA e integración	10

Tabla 15: Story Points por épica o módulo

Total = 152 SP. Con una velocidad hipotética de 24 SP/sprint, se obtienen aproximadamente 7 sprints. La velocidad no está medida en el repositorio.

4.5. Estimación con una herramienta

Aún no se ha realizado una estimación con una herramienta digital. Para completar este punto se debe registrar la herramienta usada, los parámetros de entrada, el esfuerzo, la duración, las personas y el costo; no se inventa un resultado.

4.6. Análisis de los resultados

Modelo	Tamaño	Esfuerzo referencial	Tiempo referencial	Observación
PF	465 UFP	4.650 h	29,1 PM	Muy sensible a la productividad.
PCU	169 UUCP	3.380 h	21,1 PM	Depende del peso de casos y actores.
PO	135 PO	1.080 h	6,8 PM	Más orientado a componentes.
Historias	152 SP	Depende de la velocidad	≈ 7 sprints	Útil si se mide la velocidad real.

Tabla 16: Comparación de los modelos de estimación (PM = persona-mes)

Los resultados no se deben leer como un presupuesto monetario. La ausencia de horas reales, tarifa del equipo y productividad histórica impide afirmar un costo monetario real.

Conclusiones generales

EduCampus integra un frontend Flutter Web/PWA con una API FastAPI y una base PostgreSQL. La solución cubre los módulos académicos y administrativos centrales y tiene separación de responsabilidades entre interfaz, servicios, API, validación, ORM y persistencia.

El proceso de QA queda estructurado en los niveles unitario, integración, sistema y aceptación. Hay pruebas automatizadas de arranque y responsive, mientras que las pruebas de integración, seguridad, rendimiento y aceptación requieren ejecución real y captura de evidencia.

La medición establece una línea base objetiva del repositorio. La estimación usa varios modelos con supuestos visibles y evita presentar cifras académicas como costos reales.

Finalmente, el documento distingue entre la implementación actual y sus limitaciones conocidas, de modo que lo descrito corresponde a lo que el código ejecuta hoy.